

Kaveri Stays — Database Design Assignment

Six stages and a reveal | You design the database yourself | Pure SQL

You get the old system's export and ten business rules. Designing the new schema is the assignment.

The Brief

Kaveri Stays runs three boutique hotels in South India. For eleven years they tracked every booking in a single spreadsheet. It has now collapsed under its own weight: staff have double-booked the same room twice this year, nobody can say what last quarter's occupancy was, and the guest list has the same person recorded four different ways.

They have hired you to build them a real database. You get one thing to start with: a dump of the old spreadsheet, exported as a single flat table with every column stored as text.

The business rules, in their words:

#	Rule
1	Three properties: Kaveri Riverside (Coorg), Kaveri Hilltop (Ooty), Kaveri Backwater (Alleppey). More may open later.
2	Each property has rooms. Room numbers repeat across properties — room 101 exists at all three — but must be unique within a property.
3	Rooms have a type: Standard, Deluxe or Suite. Each type has a maximum occupancy. A booking may never exceed it.
4	A room can never be booked to two guests for overlapping dates. This has already cost them money. It must be impossible, not merely discouraged.
5	A guest checking out on the 5th and another checking in on the 5th is normal and must be allowed.
6	Nightly rates vary by property, room type and season. Christmas week costs roughly double. Rates for the same room type at the same property may not have overlapping date ranges.
7	A booking moves through states: confirmed, checked in, checked out, cancelled, no-show. Cancelled and no-show bookings release the room for others.
8	A guest may pay in instalments, so one booking can have several payments. Payment methods are card, UPI, bank transfer or cash.
9	A guest may review a stay only after checking out, and only once per stay. Ratings are 1 to 5.
10	Guests are identified by email. The same person must never appear twice.

Read those ten rules again before you design anything. Eight of them translate directly into a constraint. If you find yourself planning to enforce one of them 'in the application later', you have misunderstood the assignment.

Stage 1 — Read the Wreckage

Do not write any DDL yet. Spend the first hour understanding what you have been handed.

```
createdb kaveri
psql -d kaveri -f legacy_export.sql
psql -d kaveri

SELECT * FROM legacy_reservations LIMIT 10;
```

Answer these in writing, with the SQL you used to find out:

#	Question
1.1	How many rows are in the export? How many distinct real human beings do they represent? Justify the difference.
1.2	Find every way the same guest has been recorded differently. Consider case, whitespace, and email capitalisation.
1.3	The check-in and check-out columns use three different date formats. Find all three and write a query that parses each correctly.
1.4	Some bookings list more than one room in a single row. How many? What rule of database design does this break, by name?
1.5	List every distinct value in the status column, and every distinct value in payment_method. How many genuine states are there underneath?
1.6	Missing data is represented four different ways in this file. Find all four.
1.7	The nightly_rate column is text. Find every row where it cannot be cast straight to a number, and explain why.
1.8	hotel_name, hotel_city and hotel_star repeat on every row. Which normal form does that violate, and what breaks if a hotel is upgraded from 4 to 5 stars?
1.9	Find at least one row where total_paid does not equal nightly_rate multiplied by the number of nights. Explain each discrepancy you find.
1.10	There is at least one genuine data conflict in this file: the same physical room recorded as two different room types. Find it. Decide how you will resolve it and defend your choice.

Question 1.10 has no clean answer. Real migrations always contain one. What matters is that you notice it, make a decision, and can defend it.

Stage 2 — Design and Build

Now design the real thing. Paper first, keyboard second.

#	Task
2.1	Draw an ER diagram. Every entity, every relationship, cardinality marked on both ends. Identify which relationship is many-to-many and name the bridge table.
2.2	Write the full DDL. Normalised to 3NF, with correct data types on every column. No column may be TEXT unless it genuinely holds free text.
2.3	Encode business rule 4 as a constraint. A UNIQUE constraint cannot express 'overlaps'. PostgreSQL has a constraint type that can, and it needs an extension enabled first. Find it. If your answer involves a trigger or application code, keep looking.

#	Task
2.4	Verify your solution to 2.3 still permits rule 5 (same-day turnover). Whether this works depends entirely on how you chose to represent the start and end of a stay, and whether the check-out date counts as part of it.
2.5	Make sure cancelled and no-show bookings do not block the room, while genuine double-bookings are still prevented. The constraint you used in 2.3 can be made conditional.
2.6	Encode rule 2: room numbers unique within a property, repeatable across properties.
2.7	Encode rule 3: a booking's guest count may not exceed the room type's maximum occupancy. Think carefully about where this rule can actually live.
2.8	Encode rule 6: no overlapping rate periods for the same property and room type.
2.9	Encode rule 9: one review per booking, only after checkout, rating between 1 and 5.
2.10	Encode rule 10: no duplicate guests. Decide whether you store emails lowercased or enforce case-insensitivity another way, and justify it.
2.11	Choose ON DELETE behaviour for every foreign key. For each one write a sentence explaining why CASCADE, RESTRICT or SET NULL is correct there. They should not all be the same.
2.12	Written answer: name one rule from the brief that you could not fully enforce with a constraint. Explain why, and what you would do instead.

Before you go any further — Mail your design

Save and mail your ERD and your DDL. You are about to be shown one possible answer, and the point of the next section is entirely lost if you can quietly edit your work first.

Interlude — The Reveal

Open reveal.md only now, with your Stage 2 work already committed.

reveal.md contains the table and column names that Stages 4 to 7 and the automated validator assume. It is one reasonable design, not the only one. Your job here is to compare honestly.

#	Task
R1	Confirm your Stage 2 ERD and DDL are committed and timestamped. Do not continue until they are.
R2	Open reveal.md. For every table in it, record whether you have the same thing, the same thing under a different name, or nothing equivalent.
R3	Now the reverse: list anything in your design that the reveal does not have. Explain what it is for.
R4	Where you decomposed differently, argue your case. If you put the nightly rate on the room rather than in a separate table, say why, then work out what happens when the Christmas rate arrives. If you still prefer your version, defend it.
R5	Where the reveal has something you missed, work out which of the ten business rules it exists to serve, and what would have broken without it.
R6	Rename your objects to match the reveal so that the validator and the remaining stages can run. Keep your original DDL file untouched and submit both versions.

How to read your differences

Category	What it means
Arguable	Whether reviews hang off the booking or off the guest-and-property pair. Whether rate plans live in their own table or are denormalised onto rooms. Whether payments are a separate table at all for a business this small. A well-argued alternative scores full marks here.
Not arguable	Rule 4. If overlapping bookings are not prevented by the database itself, that is wrong no matter how elegantly the rest is named. The same goes for room numbers being unique per property rather than globally.
Telling	If you find the reveal has five tables you never considered, the lesson is not that you are bad at SQL. It is that you started typing DDL before you finished reading the ten rules.

Stage 3 — Migrate and Seed

Move the old data into your new schema without losing a single booking.

#	Task
3.1	Write a migration script that reads legacy_reservations and populates every one of your tables. It must be re-runnable from scratch.
3.2	Deduplicate guests on email, case-insensitively. Pick the cleanest spelling of each name. Report how many rows became how many guests.
3.3	Normalise phone numbers to a single consistent format.
3.4	Parse all three date formats correctly. Verify no booking ended up with a check-out before its check-in.
3.5	Split multi-room rows into one booking per room. Decide how to divide the guest count and the payment between them, and justify your choice.
3.6	Map the five spellings of status onto your clean state values, and the seven spellings of payment method onto yours.
3.7	Convert every rate and total from text to a numeric type, stripping currency formatting.
3.8	Reconciliation: prove that no booking was lost. Write a query that compares the legacy row count (expanded for multi-room bookings) against your bookings table and returns zero discrepancies.
3.9	Now add data the legacy file does not contain. Each property needs its full room inventory, not just the rooms that happened to be booked. Add at least 12 rooms per property.
3.10	Build a realistic rate_plans table: at least three seasons per property per room type, with peak rates over Christmas. No overlapping periods.
3.11	Add at least 120 more bookings spread across 2025 and 2026, so occupancy figures are meaningful. Include cancellations, no-shows and repeat guests.
3.12	Add reviews for a realistic subset of completed stays. Not every guest reviews.

After 3.9 you will have rooms that have never been booked. That is intentional — several questions in Stage 4 depend on it.

Stage 4 — Answer the Business

The owner asks these questions. Every one must be a single SQL query.

Availability and operations

#	Question
4.1	Which rooms at Kaveri Backwater are free for the whole of 15–20 December 2025? This is the query the booking desk runs all day. Get it right.
4.2	Which rooms are occupied tonight, and by whom?
4.3	List every guest checking in tomorrow, with room number and nights booked.
4.4	Find any room that sat empty for more than 14 consecutive days in 2025. Report the room and the gap length.
4.5	Which rooms have never been booked at all?

#	Question
4.6	For a given date range, return each property's available room count broken down by room type.

Revenue and performance

#	Question
4.7	Total revenue per property per month for 2025.
4.8	Occupancy rate per property per month, as a percentage. Define your denominator carefully and state your definition.
4.9	ADR (average daily rate) per property: revenue divided by room-nights sold.
4.10	RevPAR (revenue per available room) per property per month. Explain in one line why RevPAR and ADR can move in opposite directions.
4.11	Which room type earns the most per available room? Not per booking — per available room.
4.12	Rank properties by revenue within each quarter, using a window function.
4.13	Running total of revenue across 2025, ordered by date.
4.14	Compare each month's revenue to the same month last year, and to the previous month. One query, using LAG.
4.15	Which bookings are not fully paid? Show the shortfall.

Guests and behaviour

#	Question
4.16	Which guests have stayed at more than one property?
4.17	Repeat guests: for each, show every stay and the gap in days since their previous one.
4.18	Split guests into four spending quartiles using NTILE. Label them Platinum, Gold, Silver, Bronze.
4.19	Cancellation and no-show rate per property. Which property has a problem?
4.20	Average review rating per property, and per room type, but only where there are at least three reviews.
4.21	Find guests who stayed but never reviewed. This is the marketing team's mailing list.
4.22	Cohort analysis: group guests by the month of their first stay, then show how many returned in each subsequent month.
4.23	For each guest, show their most recent stay only, using ROW_NUMBER.
4.24	Which single guest has generated the most revenue across all properties and all time?
4.25	Produce a revenue report per property with a grand total row, using ROLLUP.

Stage 5 — Try to Break It

Most people skip this stage. It is the one that tells you whether your constraints are real or decorative.

Attempt each of the following. Every one should be refused by the database. Paste the exact error message you received, and name the constraint that caught it. If any of them succeeds, your schema has a hole — go back to Stage 2 and fix it.

#	Attack
5.1	Book an occupied room for dates that overlap an existing confirmed booking.
5.2	Book a room with check-out before check-in.
5.3	Book a room for zero nights.
5.4	Put four guests in a room whose type has a maximum occupancy of two.
5.5	Create a second room 101 at the same property.
5.6	Insert a booking with a status of 'maybe'.
5.7	Insert a booking for a guest_id that does not exist.
5.8	Register a second guest with an existing email, differing only in capitalisation.
5.9	Delete a property that still has rooms and bookings.
5.10	Set a nightly rate to a negative number.
5.11	Leave a review with a rating of 9.
5.12	Leave two reviews on the same booking.
5.13	Leave a review on a booking that has not been checked out yet.
5.14	Create a rate plan whose dates overlap an existing plan for the same property and room type.

And these must be ALLOWED. If your schema refuses them, it is too strict:

#	Must succeed
5.15	One guest checks out on the 5th, another checks into the same room on the 5th.
5.16	A cancelled booking exists for 1–5 March; someone else books that room for 2–6 March.
5.17	Room 101 exists at all three properties simultaneously.
5.18	One booking receives three separate part-payments.

The concurrency test

Open two psql sessions side by side. This is how the real double-booking happened.

```
-- Session 1                                -- Session 2
BEGIN;                                       BEGIN;
SELECT * FROM bookings
  WHERE room_id = 1
    AND stay && daterange('2026-09-01', '2026-09-05');
-- (no rows: looks free)

-- Session 1                                -- Session 2
-- Session 1                                -- Session 2
```

```

-- same check, also sees no rows
INSERT INTO bookings ... 2026-09-01..05;
COMMIT;
INSERT INTO bookings ... 2026-09-02..06;
COMMIT;

```

#	Task
5.19	Run the sequence above. What happens at session 1's INSERT? Explain precisely why, and which constraint saved you.
5.20	Now drop your overlap constraint and run it again. Both succeed. Show the resulting double-booking. Restore the constraint.
5.21	Written answer: why is a SELECT-then-INSERT check in application code never sufficient here, no matter how carefully it is written?
5.22	Write the 'take a booking' operation as a single transaction: verify the room type allows the guest count, insert the booking, and record the deposit payment. Show what happens when it fails halfway.

Stage 6 — Make It Fast

#	Task
6.1	Run EXPLAIN ANALYZE on your availability query from 4.1. Record the plan and the execution time.
6.2	Identify the most expensive step in that plan. Is it a sequential scan? On which table, and how many rows?
6.3	Add an index that improves it. Re-run EXPLAIN ANALYZE and report the new plan and timing. State the percentage improvement.
6.4	Your overlap constraint created an index automatically. Find it in pg_indexes. What type is it, and why is that type necessary for range overlap?
6.5	Add an index supporting 'all bookings for one guest'. Prove with EXPLAIN that it is used.
6.6	Write a query where you expect an index to be used but it is not. Explain why the planner ignored it.
6.7	Run EXPLAIN ANALYZE on your occupancy query from 4.8. It is probably the slowest thing you have written. Improve it and document what you changed.
6.8	Create a materialized view for the monthly revenue report. Explain the trade-off you have just accepted, and when it must be refreshed.
6.9	Written answer: list every index you created, and for each one, name the specific query it serves and the write cost it adds.
6.10	Written answer: the owner asks for a current_occupancy column on properties, updated on every booking. Argue for or against, and describe what you would build instead.

What to Hand In

File	Contents
01_analysis.md	Your Stage 1 findings, with the SQL you used and your decision on the room-type conflict.
02_erd.png	The ER diagram as you designed it, before the reveal. Hand-drawn and photographed is fine, as long as cardinality is legible.
03_schema_original.sql	Your Stage 2 DDL exactly as committed, before you saw reveal.md. Do not tidy it.
04_reconciliation.md	The Interlude: what matched, what differed, and your arguments for anywhere you still prefer your own design.
05_schema_final.sql	The renamed schema the rest of your work runs against.
06_migration.sql	The legacy migration, including your reconciliation proof from 3.8.
07_seed.sql	The additional rooms, rate plans, bookings and reviews from 3.9 to 3.12.
08_queries.sql	All 25 Stage 4 queries, numbered, each with a one-line comment stating what it answers.
09_break_it.md	Every Stage 5 attempt, the exact error message, and the constraint that caught it. Include your validator output.
10_performance.md	Stage 6: EXPLAIN plans before and after, your index list, and your two written arguments.
README.md	How to run everything in order. Your design decisions and, importantly, anywhere you deliberately broke normalisation and why.

Marking

Stage	Marks	What is being judged
Stage 1	10	Did they actually read the data, or assume it was clean?
Stage 2	30	The heart of it. Does the schema enforce the business, or just store it?
Interlude	5	Honesty and the quality of the argument, not the number of matches.
Stage 3	10	Did every booking survive the migration? Can they prove it?
Stage 4	25	Can they answer real questions, including the hard availability query?
Stage 5	15	Validator score, plus whether they understood the concurrency test.
Stage 6	5	Evidence they can read a query plan and reason about trade-offs.

Reading the result	What it looks like
Strong signal	Reached for an exclusion constraint unprompted. Chose ON DELETE behaviour per key rather than uniformly. Noticed the room-type conflict without being told. Defended a design choice against the reveal and was right.
Weak signal	Everything is TEXT. All foreign keys CASCADE. Business rules live in comments. Availability query returns rooms that are actually occupied on the boundary dates.

Reading the result	What it looks like
Fail	Validator reports failures on the overlap test and they changed the validator. Or their committed Stage 2 schema is suspiciously identical to reveal.md.